```
from flask import Flask, request, jsonify from ultralytics import YOLO import torch import torch.nn as nn from torchvision import transforms, models from PIL import Image from flask_cors import CORS app = Flask(__name__) CORS(app) # Allow frontend to communicate with backend # --- INITIALIZE STAGE 1 (LOCATOR) --- print("Loading Stage 1: YOLOv8 Locator...") yolo_model = YOLO('best.pt') # Your Stage 1 weights # --- INITIALIZE STAGE 2 (DIAGNOSTICIAN) --- print("Loading Stage 2: MobileNetV3 Diagnostician...") device = torch.device("cuda" if torch.cuda.is_available() else "cpu") mobilenet = models.mobilenet_v3_small(weights=None) # Ensure this matches your 38 classes mobilenet.classifier[3] = nn.Linear(mobilenet.classifier[3].in_features, 38) mobilenet.load_state_dict(torch.load('best_stage2_mobilenet.pth', map_location=device)) mobilenet.to(device).eval() # Standard ImageNet transforms for MobileNetV3 crop_transform = transforms.Compose([ transforms.Resize(256), transforms.CenterCrop(224), transforms.ToTensor(), transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) ]) # Dummy class list (replace with your actual 38 class names) CLASS_NAMES = [ "Apple (Apple Scab)", "Apple (Black Rot)", "Apple (Cedar Apple Rust)", "Apple (Healthy)", "Blueberry (Healthy)", "Cherry (Powdery Mildew)", "Cherry (Healthy)", "Corn (Cercospora Leaf Spot)", "Corn (Common Rust)", "Corn (Northern Leaf Blight)", "Corn (Healthy)", "Grape (Black Rot)", "Grape (Esca / Black Measles)", "Grape (Leaf Blight)", "Grape (Healthy)", "Orange (Citrus Greening)", "Peach (Bacterial Spot)", "Peach (Healthy)", "Bell Pepper (Bacterial Spot)", "Bell Pepper (Healthy)", "Potato (Early Blight)", "Potato (Late Blight)", "Potato (Healthy)", "Raspberry (Healthy)", "Soybean (Healthy)", "Squash (Powdery Mildew)", "Strawberry (Leaf Scorch)", "Strawberry (Healthy)", "Tomato (Bacterial Spot)", "Tomato (Early Blight)", "Tomato (Late Blight)", "Tomato (Leaf Mold)", "Tomato (Septoria Leaf Spot)", "Tomato (Spider Mites)", "Tomato (Target Spot)", "Tomato (Yellow Leaf Curl Virus)", "Tomato (Mosaic Virus)", "Tomato (Healthy)" ] @app.route('/predict', methods=['POST']) def predict(): if 'image' not in request.files: return jsonify({'error': 'No image uploaded'}) file = request.files['image'] img = Image.open(file.stream).convert('RGB') # --- STAGE 1 EXECUTION --- results = yolo_model(img) if len(results[0].boxes) == 0: return jsonify({'error': 'No leaf detected by Stage 1.'}) # Get bounding box coordinates [x1, y1, x2, y2] box = results[0].boxes[0].xyxy[0].tolist() # Crop the image dynamically cropped_leaf = img.crop((box[0], box[1], box[2], box[3])) # --- STAGE 2 EXECUTION --- input_tensor = crop_transform(cropped_leaf).unsqueeze(0).to(device) with torch.no_grad(): output = mobilenet(input_tensor) probabilities = torch.nn.functional.softmax(output[0], dim=0) confidence, pred_class = torch.max(probabilities, 0) return jsonify({ 'box': box, 'disease': CLASS_NAMES[pred_class.item()], 'confidence': round(confidence.item() * 100, 2) }) if __name__ == '__main__': app.run(debug=True, port=5000)
```